

与 大语言模型

全景解读 □ 概念区分 □ 主流产品一览

适合想搞清楚「□□□□□」和「□□□」区别的人

大语言模型（[GPT-4](#)、[Claude-3](#)、[Gemini-1.5](#)，简称 GPT）本质上是一个「超级大脑」——它读了海量的书籍、代码、网页，学会了理解和生成文字。你可以把它想象成一个博学但被关在图书馆里的学者：它什么都懂，但只能通过文字跟你交流，不能自己上网、不能操作你的电脑、也不能帮你发微信。

举个例子：你问 [GPT-4](#)「深圳今天天气怎么样？」，它能给你一个基于训练数据的猜测，但它不会真正去查天气网站。因为 [GPT](#) 只是一个「推理引擎」，没有执行动作的能力。

[GPT](#) + [Tool](#)（智能体）则是「[GPT](#)」工具 + 行动能力的结合体。如果 [GPT](#) 是「大脑」，[Tool](#) 就是「有手有脚的人」——它能思考，也能行动。[GPT](#) 的核心循环是：

1. 观察（[Observe](#)）：读取环境信息，比如你的文件、网页内容、命令行输出

2. 思考（[Think](#)）：用 [GPT](#) 推理下一步该做什么

3. 行动（[Act](#)）：调用工具 [Tool](#) 执行命令、读写文件、搜索网络、发送消息

4. 反馈（[Feedback](#)）：观察行动结果，调整，继续下一步

所以，[GPT](#) 本身是「[GPT](#)」（大模型），不是「智能体」。但当你用 [GPT](#) 接入 [Tool](#) 作为底层模型时，[GPT](#) 就变成了一个「智能体」——它用 [GPT](#) 的大脑来思考，用自己的工具来执行动作。

[GPT](#) 是一个能「说」的模型，[Tool](#) 是一个能「做」的系统。[GPT](#) 提供智能，[Tool](#) 提供行动力。所有的 [Tool](#) 都依赖 [GPT](#)，但不是所有的 [Tool](#) 都能直接当 [Tool](#) 用。

二、主流 AI 代码产品

目前市面上的 AI 代码产品主要分两类：通用 AI 代码助手（什么都能干）和编码 AI 助手（专注写代码）。以下是当前主流产品：

产品名称	开发者	定位	核心特点	开源
Cursor	Cursor Labs	编码 AI 助手（IDE 插件）	终端交互式编码；支持多语言；可在 IDE 中作为子进程调用；支持本地模型；具备上下文感知、安全审查能力	否
CodeGPT	OpenAI	编码 AI 助手（API 调用）	基于 GPT 模型；命令行一键执行；支持多语言；能在 IDE 中作为编码委托工具	是
GitHub Copilot	GitHub + OpenAI	通用 AI 代码助手	支持 GPT 模型供应商；跨平台（微信、钉钉等）；持久化记忆；技能系统自我进化；定时任务；可编排多个子任务	是
Tabnine	Tabnine	AI 编码助手（IDE 插件）	深度集成 IDE；支持多语言；支持自主编码；支持本地模型双模式	否
PyCharm AI	JetBrains	AI 原生 IDE	内置 AI 模型；功能支持多文件编辑；可自主执行终端命令	部分
IntelliJ AI	JetBrains	自主编码 AI 助手	全栈自主开发；自带浏览器终端编辑器环境；适合复杂多步骤项目	否
VS Code AI	Microsoft	AI 代码助手（IDE 插件）	功能实现流式自主编码；多文件上下文感知	部分
CodeWhisperer	开源社区	编码 AI 助手（API 调用）	原生集成；可接入任何模型；轻量级；地图式代码编辑	是

□ □ □ 编码类 □ □ □ □ □ (□ □ □ □ □ □ □ □ □ □ □)

这类 □ □ □ □ □ 专门帮程序员写代码。它们的典型 workflow 是：读取项目代码 □ 理解需求 □ 修改文件 □ 运行测试 □ 提交 □ □ □。代表产品：□ □ □ □ □ □ □ □ □ □、□ □ □ □ □、□ □ □ □ □ □ □ □ □、□ □ □ □ □ □、□ □ □ □ □。

□ □ □ □ □ 生态里，这些编码 □ □ □ □ □ 都可以作为「子 □ □ □ □ □」被 □ □ □ □ □ 调遣。比如你对 □ □ □ □ □ 说「帮我重构这个项目」，□ □ □ □ □ 可能会启动一个 □ □ □ □ □ □ □ □ □ 去干这个活。

□ □ □ 通用类 □ □ □ □ □

通用 □ □ □ □ □ 不限于编码，可以做研究、数据分析、自动化任务、智能家居控制、内容创作等。代表产品：□ □ □ □ □ □ □ □ □ □、□ □ □ □ □ □ □ (早期开创者)、以及各大平台内置的 □ □ □ □ □ 助手（如微信元宝）。

□ □ □ □ □ 的独特之处在于它的「技能系统」□ □ 它能把成功完成任务的方法存成技能，下次遇到类似任务自动调用。这就像一个会自我进化的管家，越用越顺手。

三、主流大语言模型（ ）

以下按开发者分类，列出 年最具影响力的大模型。它们是 的「大脑」，不同 可以搭载不同的模型来获得不同的推理能力。

模型名称	开发者	参数规模	核心特点	开源
		未公开	多模态（文本+图像+音频）；上下文；函数调用强；推理速度快	否
		未公开	上下文；长文本理解出色；编码能力顶尖；思维链（ ）	否
		未公开	原生多模态；万 上下文（全球最长）；深度推理能力	否
	（深度求索）		国产最强开源模型；推理能力匹敌 ；价格极低；支持 上下文	是
	（阿里）	多种规格	中文能力顶尖；开源+商用版本；支持 工具调用；覆盖	部分
		多种规格	开源标杆；生态最丰富；社区微调版本众多；适合本地部署	是
			欧洲最强开源模型；支持多语言；代码和推理能力出色；上下文	是
混元+元宝	（腾讯）	未公开	中文生态深度整合；微信+元宝生态；多模态能力	否
	（月之暗面）	未公开	超长上下文（万字）；中文理解和推理出色；适合长文档分析	否
	（马斯克）	未公开	实时网络信息接入；推理模式（ ）；幽默风格	否

开源 闭源

闭源模型（GPT-4o、Claude 3.5、Gemini 2.0）：需要通过 API 调用，按 token 付费。开箱即用，性能顶级，但数据隐私和成本需要权衡。

开源模型（Llama 3.3、Qwen2.5、DeepSeek-V3）：可以下载到本地部署，数据不出门。适合对隐私敏感的企业场景，但需要自己准备算力。

小提示：Claude 3.5 支持以上所有模型，你可以今天用 GPT-4o（便宜又好用），明天换成 Qwen2.5（编码更强），随时切换，互不干扰。

模型选择建议

- 日常对话 / 中文任务：Claude 3.5、GPT-4o
- 复杂推理 / 数学：DeepSeek-V3、Qwen2.5
- 超长文档处理：Claude 3.5 (200 万 token)、GPT-4o
- 代码生成：Qwen2.5、DeepSeek-V3
- 隐私优先 / 本地部署：Qwen2.5、DeepSeek-V3、Llama 3.3
- 多模态（图像理解）：Claude 3.5、GPT-4o

四、□□□□□与□□□的关系：从「大脑」到「行动者」

□ □ □ 架构示意

一个典型的 □□□□□ 架构是这样的：

□ 用户指令 □ □ □ □ □ 控制器（编排逻辑）

□ □ □ □ □ 控制器 □ □ □ □ (推理、规划)

□ □ □ □ 返回 □ 需要调用什么工具

□ □ □ □ □ 控制器 □ 执行工具（终端、文件、网络、□ □ □ □ □）

□ 工具结果 □ □ □ □ □ 控制器 □ 再次交给 □ □ □ 分析

- 循环直到任务完成
- 返回结果给用户

可以看到，□□□在这个循环里只负责「思考和规划」，□□□□□负责「编排和执行」。两者缺一不可。

□ □ □ □ □ □ □ 框架 □ □ 单体 □ □ □ □ □

还有一种重要的区分：

□ 单体 □ □ □ □ □ (如 □ □ □ □ □ □)

□ □ □ □、□ □ □ □ □)：是具体的产品，自带模型、自带工具、开箱即用

□ □ □ □ □ 框架（如 □ □ □ □ □ □、□ □ □ □ □ □ □ □ □ □、□ □ □ □ □ □ □）：是平台 □
基础设施，让你可以接入任意模型、配置任意工具、搭建自己的 □ □ □ □ □

用造车来比喻：□□□□□□ □□□□

是一辆现成的特斯拉，你买来就能开。□□□□□□ 是一个模块化造车平台，你可以选发动机（模型）、配轮子（工具）、装导航（技能），打造最适合你的座驾。

为什么需要面向对象？

纯 LLM 有先天局限：知识截止日期固定、不能实时获取信息、不能执行操作、没有长期记忆。Agent 解决了这些问题：通过工具调用突破知识边界，通过记忆系统跨会话保持上下文，通过技能系统不断进化。这就是为什么 2023 年被称为「LLM 元年」，LLM 从「聊天」进化到「办事」。

五、快速上手建议

1. 如果你只是想体验 LLM Agent

- 在终端装个 LLM 客户端：安装 LLM 客户端，然后输入 LLM 即可
- 或者装 LLM 桌面端：安装 LLM 桌面端，体验 LLM 模式
- 想用聊天界面？装 LLM 聊天界面，体验 LLM 模式

2. 如果你想深度定制 LLM Agent

首选 LLM 框架。它能：

- 接入 LLM 模型供应商（OpenAI、Azure OpenAI、Google、Anthropic 全支持）
- 在微信、钉钉、企业微信等平台使用同一个 LLM Agent
- 写技能让它自我进化
- 定时自动执行任务
- 用 REST 协议接入外部工具（数据库、API 等）

3. 选模型还是选 LLM Agent？

一句话帮你决策：

□ 如果你需要生成文字、翻译、总结 □ 直接用

□ □ □ (□ □ □ □ □ □ □、□ □ □ □ □ □、□ □ □ □ □ □ □ □) 就够了

□ 如果你需要让 □ □ 帮你操作电脑、写代码、查实时数据、定时做任务 □ 你需要
□ □ □ □ □

□ 最好的方式：用一个通用 □ □ □ □ □ 框架（如 □ □ □ □ □ □ □），搭配最合适的
□ □ □，让两者各司其职

附录：快速参考卡片

常见问题速查

问题	答案
低代码平台是 AI 吗？	不，低代码平台是 PaaS（大模型）。用低代码平台做底层模型的工具才可能是 AI。
低代码平台是 AI 吗？	是的，低代码平台 PaaS 是一个编码 PaaS，跟低代码平台 PaaS 同类。
低代码平台是 AI 吗？	基础版低代码平台更像 PaaS 聊天界面。低代码平台的插件 PaaS 有部分 AI 特性，但不完全是 AI。
最好的编码 PaaS？	看场景：快速修 PaaS PaaS PaaS；PaaS 内无缝 PaaS PaaS PaaS PaaS；复杂多步骤 PaaS PaaS。
免费或便宜的方案？	模型用 PaaS PaaS PaaS（极便宜），低代码平台用 PaaS PaaS（开源免费），两个搭一起性价比最高。
低代码平台会取代程序员吗？	不会。低代码平台是工具，能大幅提效，但架构设计、需求理解、质量把关仍然需要人。

本文档由 PaaS PaaS 自动生成 PaaS 年 P 月 P 数据基于截至 PaaS 年 P 月的公开信息